

Speech To SQL

AI-Powered Speech-to-SQL for Interactive Database Exploration

in

medical domain

Final Project Report

Data Management Course- 2026

University of Naples Federico II (UNINA)

Prepared by

Ammar Gharaf

Amr Khalil

Mohamed Badawi

Under supervision of

Prof. Vincenzo Moscato

Dott. Francesco Di Serio

1. Introduction

Modern database systems are becoming increasingly complex, while many users interacting with them do not possess SQL knowledge or technical expertise. Traditional database querying requires understanding SQL syntax, relational schemas, joins, aggregations, and filtering operations, making database exploration difficult for non-technical users.

This project presents an AI-powered Speech-to-SQL system that allows users to interact with a relational medical database using natural spoken language. The system converts voice input into text, translates the text into executable PostgreSQL SQL queries using a Large Language Model (LLM), executes the queries against a structured medical database, and visualizes the results through an interactive dashboard.

The project follows a cascaded pipeline architecture composed of independent stages:

1. Speech Recognition (ASR)
2. Natural Language Processing
3. Text-to-SQL Translation
4. Query Execution
5. Visualization and Business Insights

Unlike end-to-end Speech-to-SQL architectures, the cascaded approach was selected because it is modular, interpretable, easier to debug, and computationally practical without requiring massive speech-to-SQL training datasets.

The project was implemented using:

- Faster-Whisper for speech recognition
- NVIDIA API with LLaMA 3.1 8B Instruct for Text-to-SQL
- PostgreSQL as the relational database
- Streamlit for the interactive dashboard
- Plotly for data visualization

Additionally, the system introduces several robustness and optimization techniques including:

- Query caching and reuse
- Rule-based normalization
- Schema-aware prompting
- SQL safety restrictions
- AI-generated business insights
- Smart visualization selection

1.1 Objectives

The objectives of the project are:

- Design and implement a normalized relational medical database
- Build a speech-to-text pipeline using ASR models
- Implement Text-to-SQL translation using LLMs
- Execute SQL queries on PostgreSQL
- Visualize results through an interactive dashboard
- Improve robustness against ASR and input inconsistencies
- Generate AI-powered business insights from query results
- Support scalable multi-user architecture

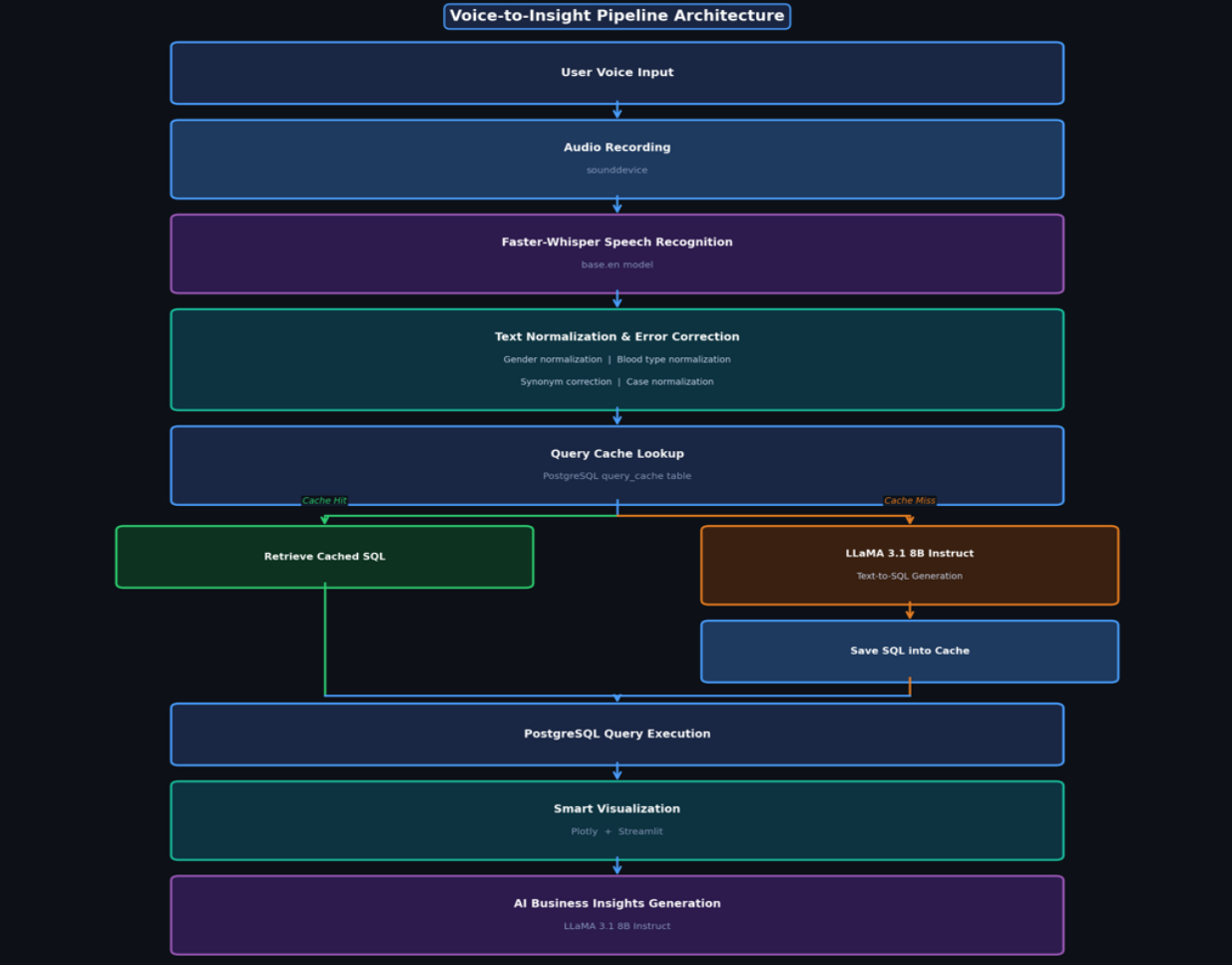
2. System Architecture

The system follows a cascaded modular pipeline architecture where each stage performs a specific task independently.

The pipeline stages are:

1. Voice Recording
2. Speech Recognition
3. Text Normalization
4. Query Cache Search
5. Text-to-SQL Generation
6. SQL Execution
7. Smart Visualization
8. AI Business Insight Generation

2.1 System Workflow



2.2 Technology Stack Summary

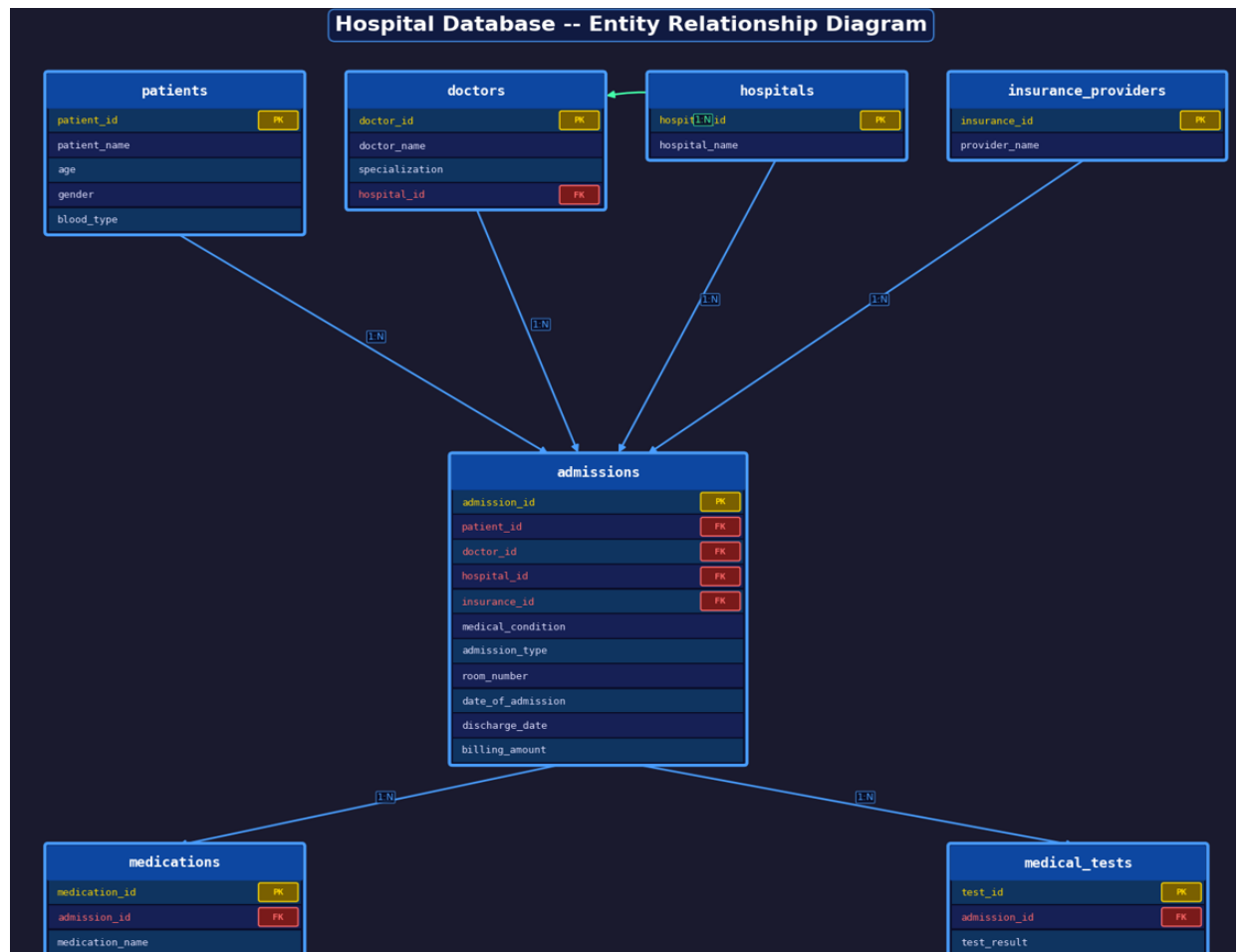
Component	Technology	Purpose
Speech Recognition	Faster-Whisper	Local ASR transcription
LLM	NVIDIA API + LLaMA 3.1 8B Instruct	Text-to-SQL generation
Database	PostgreSQL	Relational database
Dashboard	Streamlit	Interactive web application
Visualization	Plotly	Interactive charts
Database Driver	psycopg2	PostgreSQL connectivity
Programming Language	Python 3.11	System implementation
Data Processing	Pandas	Query result processing

3. Database Design

The project uses a normalized medical database designed to support complex SQL queries involving joins, aggregations, grouping, filtering, and statistical analysis.

Initially, dataset exported from [Kaggle](#) & the dataset existed as a single unstructured table containing all medical information. One of the major challenges of the project was restructuring and normalizing this data into multiple relational tables while preserving correct relationships and referential integrity.

The final schema contains seven relational tables as shown below with their relations:



3.1 SQL Query Types Demonstrated

Query Type	SQL Feature	Example Query Purpose
Simple Filter	WHERE gender = 'Female'	Count female patients
Exact Match Filtering	WHERE doctor_id = 100	Show details of doctor ID 100
Text Search	ILIKE '%cancer%'	Hospitals treating cancer patients
Multi-condition Filter	WHERE gender='Female' AND blood_type='A+'	Female patients with blood type A+

Query Type	SQL Feature	Example Query Purpose
JOIN (2 Tables)	admissions JOIN doctors	Number of appointments per doctor
JOIN (3 Tables)	admissions JOIN patients JOIN doctors	Hypertension patients and their doctors
JOIN (4 Tables)	admissions JOIN patients JOIN doctors JOIN hospitals	Repeated patients with doctor and hospital details
Aggregation	AVG(), COUNT(), SUM()	Average billing amount by hospital
GROUP BY	GROUP BY medical_condition	Count patients by medical condition
ORDER BY Ranking	ORDER BY total DESC LIMIT 10	Top 10 doctors by billing amount
DISTINCT Counting	COUNT(DISTINCT blood_type)	Count blood types and specializations
Time-based Analysis	MAX(date_of_admission) - INTERVAL	Patients admitted in last 30 days
Trend Analysis	GROUP BY date_of_admission	Admissions over time
Distribution Analysis	GROUP BY admission_type	Distribution of admission types by hospital
Comparative Analysis	GROUP BY gender	Compare billing amount by gender
Correlation Analysis	age vs hospital stay duration	Relationship between age and stay duration
Medical Analytics	JOIN medical_tests	Compare medical test results by age
Insurance Analytics	JOIN insurance_providers	Insurance provider with highest admissions
Medication Analytics	JOIN medications	Most used medications for asthma
Hospital Analytics	GROUP BY hospital_name	Hospital with highest admissions
Doctor Performance Analytics	GROUP BY doctor_name	Doctor handling most emergency admissions
Specialization Analytics	GROUP BY specialization	Doctor specialization with most patients
Blood Type Analytics	GROUP BY blood_type	Blood type with highest hypertension cases
Percentage Calculation	COUNT(*) * 100.0 / SUM(COUNT(*)) OVER()	Percentage distribution of test results
Repeat Patient Detection	COUNT(patient_id) > 1	Doctors with highest repeat patients
Average Duration Analysis	AVG(discharge_date - date_of_admission)	Average hospital stay by condition
Ranking Queries	ORDER BY avg_billing DESC	Hospital with highest average billing
Smart Comparison Queries	GROUP BY gender, test_result	Compare test results by gender
Numerical Distribution	Histogram-based aggregation	Patient age frequency

4. Stage 1 – Speech Recognition (Faster-Whisper)

The first stage converts spoken audio into text using Faster-Whisper.

Faster-Whisper is an optimized and quantized implementation of OpenAI Whisper that provides significantly faster inference speed while maintaining strong transcription accuracy.

The project uses the base.en model because it provides the best trade-off between speed and accuracy.

Although Whisper supports multilingual transcription, the English-only model was intentionally selected to improve transcription quality and reduce computational overhead.

4.1 Why Faster-Whisper Was Selected

Criterion	Faster-Whisper	Normal Whisper	Speechmatics
Deployment	Local	Local	Cloud API
Speed	Very Fast	Slower	Fast
Quantization	Yes	No	No
GPU Requirement	Optional	Heavy for large models	Cloud-based
Cost	Free	Free	Paid
Internet Required	No	No	Yes
Privacy	Fully Local	Fully Local	Audio sent to cloud
Accuracy	High	High	High
Integration	Easy Python integration	Easy	API integration
Latency Stability	Stable	Stable	Unstable depending on API

Faster-Whisper was selected because:

- It provides faster inference using quantization
- It works efficiently on CPU devices
- It avoids cloud costs
- It ensures data privacy
- It provides stable local execution

4.2 Database-Aware Normalization (DBATI-Inspired Approach)

One major challenge in cascaded Speech-to-SQL systems is ASR error propagation, where transcription mistakes negatively affect downstream SQL generation accuracy. Inspired by the DBATI (Database-Aware ASR Transcript Improvement) approach proposed in recent Speech-to-

SQL literature, this project implements a lightweight rule-based normalization layer before the Text-to-SQL stage.

The normalization layer standardizes medical and database-specific terminology, including:

- Gender normalization
- Blood type normalization
- Synonym replacement
- Medical vocabulary alignment

This database-aware preprocessing step improves SQL generation robustness by reducing ambiguity and correcting inconsistent user phrasing before passing the text to the LLM.

Although the full DBATI neural correction architecture was not implemented, the proposed normalization layer achieves a similar practical goal with significantly lower computational complexity and faster execution speed.

5. Stage 2 – Text-to-SQL Using LLaMA 3.1 8B Instruct

The second stage translates natural language into PostgreSQL SQL queries using NVIDIA-hosted LLaMA 3.1 8B Instruct.

Instead of using WrenAI or SQLCoder, LLaMA 3.1 8B Instruct was selected because it provides strong reasoning capabilities, multilingual understanding, flexible prompting, and high SQL generation quality without requiring expensive local GPU infrastructure. The model is accessed through the NVIDIA API.

5.1 Why LLaMA 3.1 8B Instruct Was Selected

Model	Advantages	Limitations
LLaMA 3.1 8B Instruct	Strong reasoning, multilingual, API-based, no local GPU needed	NVIDIA API speed instability
WrenAI	Specialized Text-to-SQL system	Requires local deployment or paid services
SQLCoder	SQL-specialized	Heavy local inference
GPT-4	Excellent reasoning	Paid and expensive

LLaMA 3.1 8B Instruct was selected because:

- It balances performance and cost
- It provides strong schema understanding
- It handles complex prompts effectively
- It works well for SQL reasoning tasks
- It avoids local GPU deployment challenges

5.2 Schema-Aware Prompting

The schema is embedded inside the prompt and includes the below to enhance the query performance:

- Table names
- Column names
- Relationships
- SQL safety rules
- Join constraints
- Aggregation instructions
- Invalid query restrictions
- Examples

This schema-aware prompting technique enables accurate SQL generation while reducing hallucinations and invalid joins.

6. Robustness Techniques

Several robustness mechanisms were implemented.

6.1 Rule-Based Text Normalization

A normalization layer was implemented before SQL generation.

The normalization system fixes:

- Gender variations
- Blood type formatting
- Medical synonyms
- Case inconsistencies
- Common ASR transcription variations

This acts as a lightweight alternative to DBATI-style correction systems.

6.2 Query Caching and Reuse

The project introduces a query caching system using an additional PostgreSQL table named:

query_cache

Previously generated SQL queries were stored alongside normalized user questions.

If the user asks the same question again:

- SQL is retrieved directly from cache

- No LLM call is required
- Response time improves significantly

This reduces API usage, latency, and computational cost.

6.3 SQL Safety and Validation

The system enforces strict SQL safety constraints.

Allowed operations SELECT ONLY & blocked others like DELETE, UPDATE, DROP and ALTER.

Additional protection includes:

- Restricting schema access
- Preventing hallucinated tables
- Preventing invalid joins
- Preventing arithmetic on text columns

7. Stage 3 – Query Execution (PostgreSQL)

Generated SQL queries are executed using PostgreSQL and psycopg2 connection pooling to make system supports:

- Concurrent users
- Efficient connection reuse
- Cached query execution
- Chunked query retrieval

PostgreSQL was selected because it provides:

- Strong relational integrity
- Foreign key enforcement
- Advanced SQL support
- High scalability
- Production-grade performance
- Ease of integration with LLAMA & Whisper

8. Stage 4 – Dashboard and Visualization

The user interface was developed using Streamlit.

The dashboard supports:

- Voice interaction
- SQL visualization
- Result tables
- Smart chart generation
- Business insights generation

Visualization is automatically selected based on:

- SQL intent
- Column types
- Aggregation detection
- Grouping patterns

Supported charts: Bar charts, Pie charts, Scatter plots, Histograms & Line chart

9. AI Business Insights Innovation

One of the major innovations of the project is the AI-powered Business Insights module.

After query execution, LLaMA 3.1 8B Instruct receives:

1. User question
2. Generated SQL query
3. Query results

The model then generates:

- Business insights
- Important findings
- Actionable recommendations

This transforms the system from a simple query interface into a lightweight Business Intelligence platform.

10. Scalability

The system was designed with scalability considerations.

Implemented scalability features include:

- PostgreSQL connection pooling
- Cached SQL reuse
- Modular pipeline architecture
- Stateless Streamlit interactions

- Efficient CPU-based ASR
- Reduced LLM calls through caching

The architecture allows multiple users to access the system simultaneously across different machines connected to the same PostgreSQL database.

11. System Evaluation

Criterion	Implementation	Status
Usability	Voice-based interaction through Streamlit dashboard with automatic SQL generation, interactive visualization, and AI insights	Strong
Accuracy	Faster-Whisper + schema-aware prompting using NVIDIA Llama 3.1 8B Instruct	Strong
Robustness	Query validation, normalization layer, SQL safety rules, retry mechanism, connection pooling, cache reuse	Strong
Visualization	Automatic smart chart selection using Plotly with table preview and KPI metrics	Good
ASR Analysis	Faster-Whisper vs Whisper vs Speechmatics comparison documented	Complete
Text-to-SQL	Comparison between Llama 3.1, WrenAI, SQLCoder, and other models documented	Complete
Pipeline Integration	Multi-stage modular cascaded pipeline with cache-aware execution path	Complete
Scalability	PostgreSQL connection pooling and multi-user concurrent execution support	Strong
Innovation	AI-generated business insights, query caching, normalization layer, smart visualization engine	Present
Database Quality	Fully normalized PostgreSQL relational schema with foreign keys and optimized joins	Strong
Performance Optimization	Faster-Whisper quantized inference, SQL caching, LRU query caching, optimized execution flow	Strong
Security & Safety	Read-only SQL generation restricted to SELECT queries with schema-aware validation	Strong
Business Intelligence Capability	LLM-generated findings, recommendations, and healthcare insights from SQL results	Good

11.1 Accuracy

The system achieves strong accuracy through the integration of Faster-Whisper for speech recognition and NVIDIA Llama 3.1 8B Instruct for Text-to-SQL generation. The schema-aware prompting strategy enables the model to correctly understand table relationships, foreign keys, and valid column usage across the healthcare database.

The generated SQL queries successfully support:

- Multi-table JOIN operations across up to 4 tables
- Aggregation queries using AVG, COUNT, SUM, and MAX
- GROUP BY and ORDER BY analysis
- Time-series and trend analysis
- Comparative analytics between genders, medical conditions, hospitals, and doctors
- Distribution analysis and ranking queries
- Complex healthcare analytical questions

Question: How many doctor specializations

Rows Returned

1

Columns

1

Visualization

TABLE

Generated SQL

```
SELECT
  (SELECT COUNT(DISTINCT specialization) FROM doctors) AS specializations;
```

Data Preview

specializations
3

Smart Visualization

Table view is the best visualization for this dataset.

AI Business Insights

Business Insight: There are 6 distinct specializations among the doctors.

Important Finding: To determine which specialization has the highest number of doctors, we need to analyze the count of doctors for each specialization. However, the current query only provides the count of unique specializations, not the count of doctors per specialization.

Actionable Recommendation: To answer the user's question, we need to modify the SQL query to include the count of doctors for each specialization. We can use a query like this:

```
SELECT
  specialization,
  COUNT(*) AS doctor_count
FROM
  doctors
GROUP BY
  specialization
ORDER BY
  doctor_count DESC;
```

This query will provide the count of doctors for each specialization, allowing us to identify the specialization with the highest number of doctors.

12. Limitations

The project has several limitations.

12.1 NVIDIA API Latency

NVIDIA API response times sometimes unstable some requests execute quickly and others high latency.

12.2 Local LLM Deployment Constraints

Running large LLMs locally requires significant GPU resources and increases inference time.

12.3 Dataset Complexity

The original dataset was highly unstructured and existed as a single table. A significant amount of work was required to normalize the dataset and establish relational connections.

13. Future Improvements

Future improvements include:

A) Semantic Query Search

Instead of exact-match query caching, semantic similarity search can be implemented using embeddings.

B) LLM-Based Chart Selection

Future versions may allow LLM to select optimal chart types based on User intent, SQL query & Result structure

C) Advanced ASR Error Correction

Implementing database-aware transcript correction inspired by DBATI.

D) Cloud Deployment

Migrating the system to cloud-hosted PostgreSQL and scalable API infrastructure.

14. Conclusion

Project demonstrates a complete AI-powered Speech-to-SQL system for medical database exploration as the system integrates Faster-Whisper ASR, LLAMA3.1, PostgreSQL, Streamli & Plotly into a coherent end-to-end pipeline capable of transforming spoken natural language into executable SQL queries and interactive business insights.